



RV College of
Engineering®

Go, change the world®

UNIT 3

HIVE



Hive

- Apache Hive is a distributed data warehouse software built on top of Apache Hadoop for querying and managing large datasets stored in HDFS.
- **Key aspects :**
 - **Data Structuring:** Hive adds structure to unstructured data in HDFS by creating databases and tables, enabling data warehouse functionality.
 - **HiveQL (HQL):** A SQL-like query language that makes it easy for users familiar with SQL to process data on Hadoop.
 - **Metastore:** A central repository that stores metadata (table schema, data location) about Hive tables, typically using a relational database.
 - **Architecture:** Key components include the User Interface (CLI/Web), JDBC/ODBC Drivers, Compiler, and Execution Engine.



RV College of
Engineering®

Go, change the world®

Components of Hive Architecture

- **UI:** User interface (Beeline, Web UI).
- **Metastore:** Stores data warehouse metadata.
- **Driver:** Manages HiveQL statements, performing analysis, compilation, and optimization.
- **Execution Engine:** Executes tasks created by the compiler.



Configuring Hive

Go, change the world®

- Hive is configured using an XML configuration file like Hadoop's. The file is called *hivesite*.
- *xml* and is located in Hive's *conf* directory. This file is where you can set properties that you want to set every time you run Hive. The same directory contains *hivedefault*.
- *xml*, which documents the properties that Hive exposes and their default values.
- We can override the configuration directory that Hive looks for in *hive-site.xml* by passing the `--config` option to the hive command:

```
% hive --config /Users/tom/dev/hive-conf
```



RV College of
Engineering®

Go, change the world®

- The *hive-site.xml* file is a natural place to put the cluster connection details: you can specify the filesystem and resource manager using the usual Hadoop properties, `fs.defaultFS` and `yarn.resourcemanager.address`.



Configuration Properties of Hadoop

- **Execution engines** - Hive was originally written to use MapReduce as its execution engine, and that is still the default. It is now also possible to run Hive using Apache Tez as its execution engine, and work is underway to support Spark.
- Both Tez and Spark are general directed acyclic graph (DAG) engines that offer more flexibility and higher performance than MapReduce.
- **Logging**- You can find Hive's error log on the local filesystem at `${java.io.tmpdir}/${user.name}/hive.log`. It can be very useful when trying to diagnose configuration problems or other types of error. Hadoop's MapReduce task logs are also a useful resource for troubleshooting.



- On many systems, `${java.io.tmpdir}` is `/tmp`, but if it's not, or if you want to set the logging directory to be another location, then use the following:
% hive -hiveconf hive.log.dir='/tmp/\${user.name}'
- The logging configuration is in `conf/hive-log4j.properties`, and you can edit this file to change log levels and other logging-related settings.



Hive Services

- The Hive shell is only one of several services that you can run using the hive command.
- We can specify the service to run using the --service option.
- hive --service help to get a list of available service names; some of the most useful ones are described in the following list:
- **cli**-The command-line interface to Hive (the shell). This is the default service.
- **Hiveserver2**-Runs Hive as a server exposing a Thrift service, enabling access from a range of clients written in different languages.
- **Beeline** - A command-line interface to Hive that works in embedded mode (like the regular CLI), or by connecting to a HiveServer 2 process using JDBC.

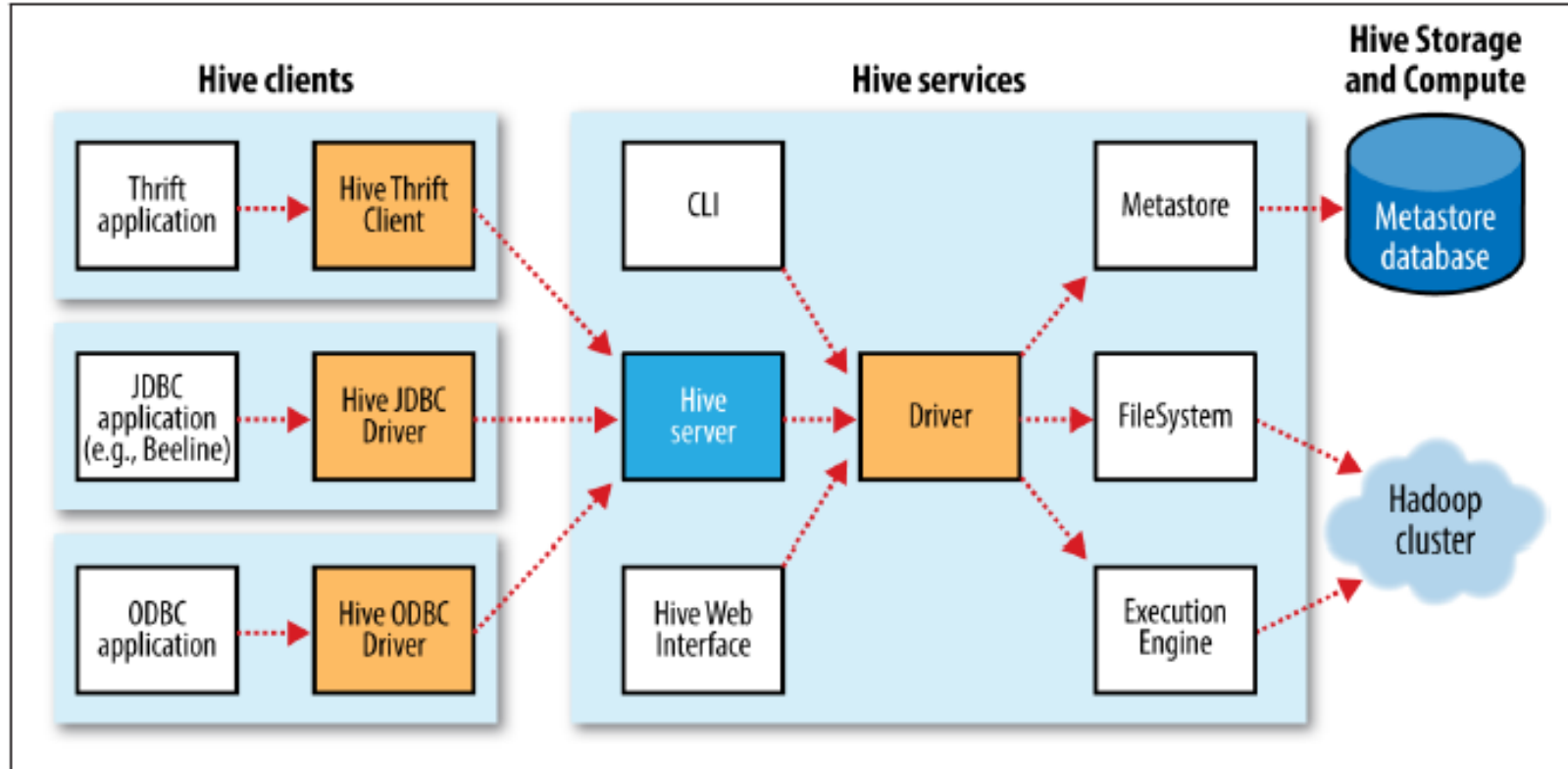


- **Hwi** - The Hive Web Interface. A simple web interface that can be used as an alternative to the CLI without having to install any client software.
- **Jar** - The Hive equivalent of hadoop jar, a convenient way to run Java applications that includes both Hadoop and Hive classes on the classpath.
- **Metastore** - By default, the metastore is run in the same process as the Hive service. Using this service, it is possible to run the metastore as a standalone (remote) process.



Hive Architecture

- **Hive clients** - If you run Hive as a server (hive --service hiveserver2), there are a number of different mechanisms for connecting to it from applications.
- **Thrift Client** - The Hive server is exposed as a Thrift service, so it's possible to interact with it using any programming language that supports Thrift.
- **JDBC driver** - Hive provides a Type 4 (pure Java) JDBC driver, defined in the class `org.apache.hadoop.hive.jdbc.HiveDriver`.
- When configured with a JDBC URI of the form `jdbc:hive2://host:port/dbname`, a Java application will connect to a Hive server running in a separate process at the given host and port.
- **ODBC driver** - An ODBC driver allows applications that support the ODBC protocol (such as business intelligence software) to connect to Hive.





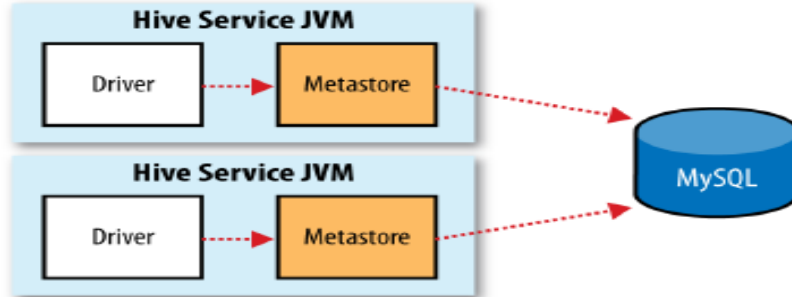
The Meta Store

- The *metastore* is the central repository of Hive metadata.
- The metastore is divided into two pieces: a service and the backing store for the data.
- By default, the metastore service runs in the same JVM as the Hive service and contains an embedded Derby database instance backed by the local disk. This is called the *embedded metastore* configuration.
- Using an embedded metastore is a simple way to get started with Hive; however, only one embedded Derby database can access the database files on disk at any one time, which means you can have only one Hive session open at a time that accesses the same metastore.
- Trying to start a second session produces an error when it attempts to open a connection to the metastore.

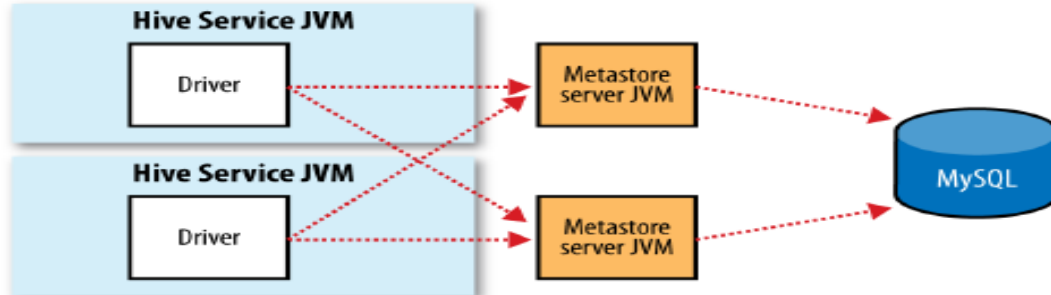
**Embedded
metastore**



**Local
metastore**



**Remote
metastore**





Comparison with Traditional Databases

- **Schema on Read Versus Schema on Write:**
- In a traditional database, a table's schema is enforced at data load time. If the data being loaded doesn't conform to the schema, then it is rejected. This design is sometimes called *schema on write* because the data is checked against the schema when it is written into the database.
- Hive, on the other hand, doesn't verify the data when it is loaded, but rather when a query is issued. This is called *schema on read*.



Updates, Transactions, and Indexes

- Updates, transactions, and indexes are mainstays of traditional databases.
- Hive has long supported adding new rows in bulk to an existing table by using INSERT INTO to add new data files to a table.
- From release 0.14.0, finer-grained changes are possible, so you can call INSERT INTO TABLE...VALUES to insert small batches of values computed in SQL.
- In addition, it is possible to UPDATE and DELETE rows in a table.
- Hive also has support for table- and partition-level locking.
- Hive indexes can speed up queries in certain cases. A query such as SELECT * from t WHERE x = a, for example, can take advantage of an index on column x, since only a small portion of the table's files need to be scanned.



- Hive's SQL dialect, called HiveQL, is a mixture of SQL-92, MySQL, and Oracle's SQL dialect.

Table 17-2. A high-level comparison of SQL and HiveQL

Feature	SQL	HiveQL	References
Updates	UPDATE, INSERT, DELETE	UPDATE, INSERT, DELETE	"Inserts" on page 500; "Updates, Transactions, and Indexes" on page 483
Transactions	Supported	Limited support	
Indexes	Supported	Supported	
Data types	Integral, floating-point, fixed-point, text and binary strings, temporal	Boolean, integral, floating-point, fixed-point, text and binary strings, temporal, array, map, struct	"Data Types" on page 486
Functions	Hundreds of built-in functions	Hundreds of built-in functions	"Operators and Functions" on page 488
Multitable inserts	Not supported	Supported	"Multitable insert" on page 501
CREATE TABLE...AS SELECT	Not valid SQL-92, but found in some databases	Supported	"CREATE TABLE...AS SELECT" on page 501
SELECT	SQL-92	SQL-92. SORT BY for partial ordering, LIMIT to limit number of rows returned	"Querying Data" on page 503
Joins	SQL-92, or variants (join tables in the FROM clause, join condition in the WHERE clause)	Inner joins, outer joins, semi joins, map joins, cross joins	"Joins" on page 505
Subqueries	In any clause (correlated or noncorrelated)	In the FROM, WHERE, or HAVING clauses (uncorrelated subqueries not supported)	"Subqueries" on page 508
Views	Updatable (materialized or nonmaterialized)	Read-only (materialized views not supported)	"Views" on page 509



- Hive supports both primitive and complex data types.
- Primitives include numeric, Boolean, string, and timestamp types.
- The complex data types include arrays, maps, and structs.
- **Primitive types** - Hive's primitive types correspond roughly to Java's, although some names are influenced by MySQL's type names (some of which, in turn, overlap with SQL-92's).
- There is a BOOLEAN type for storing true and false values. There are four signed integral types: TINYINT, SMALLINT, INT, and BIGINT, which are equivalent to Java's byte, short, int, and long primitive types, respectively



Operators and Functions

- The usual set of SQL operators is provided by Hive: relational operators (such as $x = 'a'$ for testing equality, $x \text{ IS NULL}$ for testing nullity, and $x \text{ LIKE } 'a\%'$ for pattern matching), arithmetic operators (such as $x + 1$ for addition), and logical operators (such as $x \text{ OR } y$ for logical OR).
- You can retrieve a list of functions from the Hive shell by typing SHOW FUNCTIONS.4 To get brief usage instructions for a particular function, use the DESCRIBE command:

hive> **DESCRIBE FUNCTION length;**

length(str | binary) - Returns the length of str or number of bytes in binary data.



Tables

- A Hive table is logically made up of the data being stored and the associated metadata describing the layout of the data in the table.
- The data typically resides in HDFS, although it may reside in any Hadoop filesystem, including the local filesystem or S3.

Managed Tables and External Tables

- *external table*, which tells Hive to refer to the data that is at an existing location outside the warehouse directory.
- The difference between the two table types is seen in the LOAD and DROP semantics.



- When you load data into a managed table, it is moved into Hive's warehouse directory.

For example, this:

```
CREATE TABLE managed_table (dummy STRING);
```

LOAD DATA INPATH '/user/tom/data.txt' INTO table managed_table; will
move the file *hdfs://user/tom/data.txt* into Hive's warehouse directory for the

- **managed_table table**, which is *hdfs://user/hive/warehouse/managed_table*.
- When you drop an external table, Hive will leave the data untouched and only delete the metadata.



Partitions and Buckets

- Hive organizes tables into *partitions*—a way of dividing a table into coarse-grained parts based on the value of a *partition column*, such as a date.
- Tables or partitions may be subdivided further into *buckets* to give extra structure to the data that may be used for more efficient queries.
- For example, bucketing by user ID means we can quickly evaluate a user-based query by running it on a randomized sample of the total set of users.



Partitions and Buckets

- Partitions are defined at table creation time using the PARTITIONED BY clause,⁷ which takes a list of column definitions.
- For the hypothetical logfiles example, we might define a table with records comprising a timestamp and the log line itself:

```
CREATE TABLE logs (ts BIGINT, line STRING)
```

```
PARTITIONED BY (dt STRING, country STRING);
```

When we load data into a partitioned table, the partition values are specified explicitly:

```
LOAD DATA LOCAL INPATH 'input/hive/partitions/file1'
```

```
INTO TABLE logs
```

```
PARTITION (dt='2001-01-01', country='GB');
```



Partitions and Buckets

- The data within a bucket may additionally be sorted by one or more columns. This allows even more efficient map-side joins, since the join of each bucket becomes an efficient merge sort. The syntax for declaring that a table has sorted buckets is:

```
CREATE TABLE bucketed_users (id INT, name STRING)  
CLUSTERED BY (id) SORTED BY (id ASC) INTO 4 BUCKETS;
```



Storage Formats

- There are two dimensions that govern table storage in Hive: the *row format* and the *file format*.
- The row format dictates how rows, and the fields in a particular row, are stored.
- The file format dictates the container format for fields in a row.
- The simplest format is a plain-text file, but there are row-oriented and column-oriented binary formats available, too.



Importing Data

- If we want to import data from a relational database directly into Hive, have a look at Sqoop.

Inserts

Here's an example of an INSERT statement:

```
INSERT OVERWRITE TABLE target  
SELECT col1, col2  
FROM source;
```

For partitioned tables, you can specify the partition to insert into by supplying a PARTITION clause:

```
INSERT OVERWRITE TABLE target  
PARTITION (dt='2001-01-01')  
SELECT col1, col2  
FROM source;
```

You can specify the partition dynamically by determining the partition value from the SELECT statement:

```
INSERT OVERWRITE TABLE target  
PARTITION (dt)  
SELECT col1, col2, dt  
FROM source;
```

This is known as a *dynamic partition insert*.



Multitable insert

Go, change the world®

In HiveQL, you can turn the INSERT statement around and start with the FROM clause for the same effect:

```
FROM source  
INSERT OVERWRITE TABLE target  
  SELECT col1, col2;
```

Here's an example that computes various statistics over the weather dataset:

```
FROM records2  
INSERT OVERWRITE TABLE stations_by_year  
  SELECT year, COUNT(DISTINCT station)  
  GROUP BY year  
INSERT OVERWRITE TABLE records_by_year  
  SELECT year, COUNT(1)  
  GROUP BY year  
INSERT OVERWRITE TABLE good_records_by_year  
  SELECT year, COUNT(1)  
  WHERE temperature != 9999 AND quality IN (0, 1, 4, 5, 9)  
  GROUP BY year;
```

There is a single source table (records2), but three tables to hold the results from three different queries over the source.



Altering Tables and Dropping Tables *Go, change the world®*

- You can rename a table using the ALTER TABLE statement:
`ALTER TABLE source RENAME TO target;`
- The DROP TABLE statement deletes the data and metadata for a table. In the case of external tables, only the metadata is deleted; the data is left untouched.
- If you want to delete all the data in a table but keep the table definition, use TRUNCATE TABLE. For example:
`TRUNCATE TABLE my_table;`
- This doesn't work for external tables; instead, use `dfs -rmr` (from the Hive shell) to remove the external table directory directly.



Querying the Data – Sorting and Aggregating

- Sorting data in Hive can be achieved by using a standard ORDER BY clause. ORDER BY performs a parallel total sort of the input.
- BY produces a sorted file per reducer.

1

```
hive> FROM records2
> SELECT year, temperature
> DISTRIBUTE BY year
> SORT BY year ASC, temperature DESC;
1949      111
1949      78
1950      22
1950       0
1950     -11
```



- Using an approach like Hadoop Streaming, the TRANSFORM, MAP, and REDUCE clauses make it possible to invoke an external script or program from Hive.

Example 17-1. Python script to filter out poor-quality weather records

```
#!/usr/bin/env python
```

```
import re
import sys
```

```
for line in sys.stdin:
    (year, temp, q) = line.strip().split()
    if (temp != "9999" and re.match("[01459]", q)):
        print "%s\t%s" % (year, temp)
```

We can use the script as follows:

```
hive> ADD FILE /Users/tom/book-workspace/hadoop-book/ch17-hive/
src/main/python/is_good_quality.py;
```

```
hive> FROM records2
> SELECT TRANSFORM(year, temperature, quality)
> USING 'is_good_quality.py'
> AS year, temperature;
```

```
1950    0
1950    22
1950   -11
1949   111
1949    78
```



- One of the nice things about using Hive, rather than raw MapReduce, is that Hive makes performing commonly used operations very simple. Join operations are a case in point, given how involved they are to implement in MapReduce.

Inner joins

The simplest kind of join is the inner join, where each match in the input tables results in a row in the output. Consider two small demonstration tables, `sales` (which lists the names of people and the IDs of the items they bought) and `things` (which lists the item IDs and their names):

```
hive> SELECT * FROM sales;
Joe      2
Hank     4
Ali      0
Eve      3
Hank     2
hive> SELECT * FROM things;
2      Tie
4      Coat
3      Hat
1      Scarf
```

We can perform an inner join on the two tables as follows:

```
hive> SELECT sales.*, things.*
> FROM sales JOIN things ON (sales.id = things.id);
Joe      2      2      Tie
Hank     4      4      Coat
Eve      3      3      Hat
Hank     2      2      Tie
```



RV College of
Engineering®

Go, change the world®

- Kindly study the remaining topics from textbook .